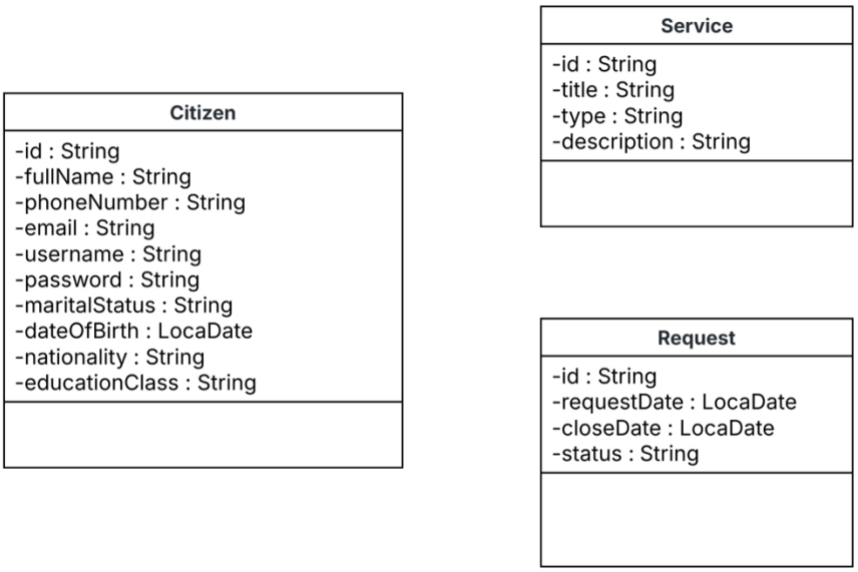
**Shahad Almalki**

# Absher

Absher is an official digital platform that provides citizens and residents with access to a wide range of government services in a secure and electronic way, with no need to visit physical offices.
I focused on  three main models: The Citizen model represents individuals who use the system, the Service model represents the available government services, and the Request model represents the requests that citizens apply for a specific service and tracks their status.

**Class diagram**:



**Citizen model has**: id, fullName, phoneNumber, email, username password, maritalStatus, dateOfBirth, nationality, educationClass

**Validation:**

ID:

- can't be null
- Must start with 1
- Length should be exactly 10 digits

Syntax:

```
@NotEmpty(message = "National ID cannot be null")
@Pattern(regexp = "^1\\d{9}$", message = "National ID must start with
1and must be 10 digits")
private String id;
```

FullName:

- can't be null
- Must have only letters and at least two words
- Length must be more than 8 letters and less than 51 letters

Syntax:

```
@NotEmpty(message = "Full name cannot be null")
@Pattern(regexp = "^[A-Za-z]+( [A-Za-z]+)+$",
message = "Full name must contain only letters and at least two words")
@Size(min = 9, max = 50)
private String fullName;
```

PhoneNumber:

- can't be null
- Must start with 05 and be 10 digits

Syntax:

```
@NotEmpty(message = "Phone number cannot be null")
@Pattern(regexp = "^05\\d{8}$", message = "Phone number must start
with 05 and be 10 digits ")
private String phoneNumber;
```

Email:

- can't be null Length must
- Must be valid email

Syntax:

```
@NotEmpty(message = "Email cannot be null")
@Email(message = "Email must be valid")
private String email;
```

Username:

- can’t be null
- Minimum 8 characters
- Start with a letter
- No special characters
- Must include lowercase letters
- Must include numbers

Syntax:

```
@NotEmpty(message = "Username cannot be null")
@Pattern( regexp = "^(?=.*[a-z])(?=.*\\d)[A-Za-z][A-Za-z\\d]{7,}$",
message = "Username must be at least 8 characters, start with a
letter, include lowercase letters and numbers, and contain no
special characters")
private String username;
```

Password:

- can’t be null
- Minimum 8 characters
- Must include uppercase letter
- Must include lowercase letter
- Must include number

Syntax:

```
@NotEmpty(message = "Password cannot be null")
@Pattern(regexp = "^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d).{8,}$", message =
"Password must be at least 8 characters and include uppercase,
lowercase, and a number")
private String password;
```

MaritalStatus:

- can’t be null
- Must be one of these single,married,divorced or widowed

Syntax:

```
@NotEmpty(message = "Marital status cannot be null")
@Pattern(regexp = "(?i)^(single|married|divorced|widowed)$",
message = "Invalid marital status")
private String maritalStatus;
```

DateOfBirth:

- can’t be null
- Must be in the past

Syntax:

```
@NotNull(message = "Date of birth cannot be null")
@Past(message = "Date of birth must be in the past")
private LocalDate dateOfBirth;
```

Nationality:

- can't be null

Syntax:

```
@NotEmpty(message = "Nationality cannot be null")
private String nationality;
```

EducationClass:

- can't be null
- Must be one of these Primary, Intermediate, Secondary, Diploma, Bachelor, Master or PhD

Syntax:

```
@NotEmpty(message = "Education class cannot be null")
@Pattern(regexp =
"(?i)^(Primary|Intermediate|Secondary|Diploma|Bachelor|Master|Ph
D)$", message = "Invalid education class")
private String educationClass;
```

**Service model** has: id, title, type, description

**Validation**:

ID:

- can't be null
- Must be more than 3 characters

Syntax:

```
@NotEmpty(message = "ID cannot be null")
@Size(min = 4)
private String id;
```

Title:

- can't be null
- Must be more than 2 characters and less than 51

Syntax:

```
@NotEmpty(message = "Title cannot be null")
@Size(min = 3, max = 50)
```

```
private String title;
```

Type:

- can’t be null
- Must be one of the allowed government services

Syntax:

```
@NotEmpty(message = "Type cannot be null")
@Pattern(regexp = "(?i)^(Postal Delivery Service|Civil Affairs|Passports|Traffic|Public Security|General Services|Care Of Martyrs And Injured Families|General Directorate Of Prisons|Public Prosecution Services|Emirates And Regions|Border Guard)$", message = "Invalid service type")
private String type;
```

Description:

- can’t be null
- Must be more than 50 characters

```
@NotEmpty(message = "Description cannot be null")
@Size(min = 50)
private String description;
```

**Request model** has: id, citizen, service, requestDate, closeDate, status

**Validation**:

ID:

- can’t be null
- Must be more than 3 characters

Syntax:

```
@NotEmpty(message = "ID cannot be null")
@Size(min = 4)
private String id;
```

Citizen:

- can’t be null

Syntax:

```
@NotNull(message = "Citizen cannot be null")
private Citizen citizen;
```

Service:

- can’t be null

Syntax:

```
@NotNull(message = "Service cannot be null")
private Service service;
```

RequestDate:

- can’t be null
- Must be present of past

Syntax:

```
@NotNull(message = "Request date cannot be null")
@PastOrPresent(message = "Request date cannot be in the future")
private LocalDate requestDate;
```

CloseDate:

- Must be present or future

Syntax:

```
@FutureOrPresent(message = "Close date must be today or in the
future")
private LocalDate closeDate;
```

Status :

- can’t be null
- must be one of these Pending, Approved, Rejected or Closed

Syntax:

```
@NotEmpty(message = "Status cannot be null")
@Pattern(regexp = "(?i)^(Pending|Approved|Rejected|Closed)$",
message = "Invalid status")
private String status;
```